

Basic Elements of Prolog

1. Facts

Each **fact** consists of a **predicate** followed by its **arguments** in parentheses.

Predicate names (rule names) must start with a lowercase letter

Example:

```
parent(alice, bob) .           % Alice is a parent of  
Bob
```

```
parent(bob, charlie) .        % Bob is a parent of  
Charlie
```

```
male(bob) .                   % Bob is male
```

```
female(alice) .               % Alice is female
```

2. Rules

A **rule** consists of a **head** and a **body**, separated by :- (if).

Variables (X, Y, Z) start with an **uppercase letter**.

Example.

```
child(X, Y):-parent(Y, X).
```

Example.

```
?-[user] .
```

```
parent(alice, bob) .
```

```
parent(bob, charlie) .
```

```
male(bob) .
```

```
female(alice) .
```

```
ctrl+D
```

(on Windows, if ctrl+D doesn't work, use **end_of_file.** as the end-of-file character.)

3. Queries

Queries allow users to **ask questions** and **retrieve information** from the Prolog knowledge base.

Example Queries:

```
?- parent(X, bob) .
```

```
X = alice. (Alice is Bob's parent)
```

```
?- male(bob) .
```

true

?- indicates a query.

4. Arithmetic

Prolog supports basic arithmetic operations.

Example:

```
sum(X, Y, Z) :- Z is X + Y .
```

Query:

```
?- sum(3, 5, Result) .
```

```
% Answer: Result = 8
```

4. Unification

Example:

```
?- parent(alice, X) .
```

Output: X = bob.

5. Inference Mechanisms

Modus ponens

if $A \rightarrow B$ and A is true, then B must be true.

```
child(X, Y):-parent(Y, X).
```

Query:

```
?- child(bob, alice).
```

Output:

```
true.
```

Forward Chaining

Example.

```
?-[user] .
```

```
flu(X) :- fever(X), cough(X) .
```

```
takeRest(X) :- flu(X) .
```

```
fever(john) .
```

```
cough(john) .
```

```
ctrl+D
```

(on Windows, if ctrl+D doesn't work, use **end_of_file.** as the end-of-file character.)

Query:

```
?- takeRest(john) .
```

Output:

```
true
```

Backward Chaining

Example.

```
?-[user] .  
flu(X) :- fever(X), cough(X) .  
fever(X) :- chills(X) .  
cough(john) .  
chills(john) .  
ctrl+D
```

(on Windows, if ctrl+D doesn't work, use **end_of_file.** as the end-of-file character.)

Query:

```
?- flu(john) .
```

Output:

true

Explanation:

1. flu(john) calls fever(john) and cough(john).
2. cough(john) is found in KB.
3. fever(john) calls chills(john), which is found in KB.
4. Since all sub-goals are satisfied, flu(john) is true.

.

Example: Grandparent Relationship

```
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

```
?- grandparent(alice, charlie) .
```

Output:

true

Example:

```
ancestor(X, Y) :- parent(X, Y) .
```

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y) .
```

Query:

```
?- ancestor(alice, charlie) .
```

Output: true.

Example: Finding Siblings

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

Query:

```
?- sibling(bob, charlie).
```

Output:

true

Explanation:

- Prolog searches for a common parent of bob and charlie.
- parent(alice, bob) and parent(bob, charlie) do not share the same parent.
- Prolog backtracks and tries other rules.
- If no solution is found, it returns false.